Carver AI

Multi-Angle Multimodal / Geometry-Conditioned Generation Audit

Implementation plan based on the repository as audited at commit **56acf8c**. This is an English translation and structured restatement of the code audit.

Prepared for Nghia IT

1. Executive summary

• Carver already has graph-aware generation, stable asset IDs, queue-backed workers, OpenAI image editing, idempotent job creation, and provider fakes.

• The central missing layer is a provider-independent conditioning package that combines source identity, normalized camera, evidence images, masks, references, and semantic instructions.

• Orbit currently drops rawGoal, operations, reference roles, and most constraints when its special prompt path returns early.

• Plan still falls through the generic compiler; targetHeight and viewDirection can disappear from the provider prompt.

• The Three.js preview is useful authoring UI, but it is not yet a calibrated geometry engine: Plan and Orbit use different axes and virtual scales.

• Worker retries can repeat already-paid multi-angle calls because all shots are generated before outputs are persisted.

• The first geometry MVP should use one canonical source, a normalized camera, coarse target-view RGB guidance, and a target-space uncertainty map.

• Depth should initially be an internal worker input used to create a guide, not a provider-native field.

• Do not start with full 3D, Blender, dense segmentation, or a paid VLM evaluator.

Evidence boundary. Existing tests pass, but they do not prove geometry preservation. Targeted probes found the semantic losses and replay behavior documented below.

2. Current pipeline

```
flowchart TD
  UI[MultiAngleModal / CameraShotSet] --> STATE[CanvasCameraShotSetState]
  STATE --> GRAPH[Graph context resolution]
  GRAPH --> JOB[POST ai-jobs]
  JOB --> DB[ai_jobs payload + checkpoint + credit ledger]
  DB --> QUEUE[BullMQ jobId]
  QUEUE --> WORKER[Worker loads authoritative job]
  WORKER --> PROMPT[Orbit compiler or generic Plan compiler]
  WORKER --> SOURCES[Source + references + optional edit mask]
  PROMPT --> OPENAI[OpenAI Images Edit: prompt + image[] + mask]
  SOURCES --> OPENAI
  OPENAI --> PERSIST[R2 + assets rows]
  PERSIST --> RESULT[Job result + gateway URLs + canvas outputs]
```

The UI stores **CanvasMultiAngleCamera** with both Plan and Orbit transforms. The shared boundary is **CameraShotDirective**. The worker receives it through **CameraShotGenerationContext**, while the provider receives only bytes and a prompt.

3. Target pipeline

```
flowchart TD
  GRAPH[Canvas inbound graph] --> REQUEST[GenerationRequest v1]
  REQUEST --> EVIDENCE[SceneEvidenceBuilder]
  REQUEST --> SEMANTIC[OrbitPromptCompiler / PlanPromptCompiler]
  EVIDENCE --> CONDITION[ModelConditioning v1]
  SEMANTIC --> CONDITION
  CONDITION --> ADAPTER[Provider capability adapter]
  ADAPTER --> MODEL[Image model]
  MODEL --> CHECKPOINT[Durable candidate checkpoint]
  CHECKPOINT --> EVAL[Geometry + visual deterministic evaluator]
  EVAL --> SELECT[Rerank and persist selected shot]
  SELECT --> CANVAS[Asset gateway + canvas result]
```

The key design rule is that prompt, visual conditioning, and geometric conditioning remain separate inputs. A provider adapter may convert geometry to images, but the shared contract must not contain GPT-specific multipart fields.

4. Current-state findings

4.1 Trace and data loss

Stage	What exists	What is lost or ambiguous
Camera UI	Plan u/v/height/target/lens/pitch/roll; Orbit rotate/tilt/distance/lens	Different axes, virtual scale, no calibrated source intrinsics
Graph	Inbound edges resolve source and reference assets	Reference roles are not preserved alongside resolved buffers
BFF/job	Snapshot validation, source edge check, order check, idempotency	Hash includes runtime URLs; normalized camera is absent
Prompt	PromptPlanV2 has operations, constraints, provenance	Orbit early return discards raw goal, operations, roles, hard constraints
Worker	Per-shot prompt and image resolution	All shots wait before persistence; no evidence builder
Provider	Multipart image[] and optional mask; n for ordinary generation	No camera matrix, depth, warp, uncertainty, or role manifest
Persistence	Stable asset IDs, R2, output gallery, camera label/order	No per-shot conditioning hash, candidate identity, evidence or score

4.2 Verified bugs

• **F01:** packages/ai/src/prompt-engine/compileProviderPrompt.ts returns Orbit prose from buildChangeAnglePrompt() and omits rawGoal, operations, references and most constraints.

• **F02:** buildNovelViewAdditionalInstruction() only carries a small preserve subset; protected region/object locks do not reach the special prompt.

• **F03:** Plan targetHeight and viewDirection do not affect the provider prompt in the current compiler.

• **F04:** generation-service.ts does not stop before the provider when the compiled decision is reject or require_review.

• **F05:** executeGeneratedImageJob() accumulates all shot batches before persistence, making partial failure expensive to replay.

• **F06:** createRetryIdempotencyKey() is reached after the RPC path that can already return the existing job; the terminal rerun branch is ineffective.

• **F07:** OpenAI edit FormData uses OPENAI_IMAGE_MODEL instead of the resolved model parameter.

• **F08:** reference resolver truncates to four and drops role/context metadata from returned image buffers.

• **F09:** sourceImageId can fall back to a node ID even though the field is intended to identify an image asset.

Probe results: Orbit prompt did not contain rawGoal, operation, reference, or protected-region sentinels. Changing Plan targetHeight from 0 to 10 and viewDirection still produced the same prompt. A two-shot failure caused both shots to be called again on replay. A requested dated GPT Image model was sent as the constant gpt-image-2.

5. Gap analysis and priorities

Capability	Current	Target	Gap / change	Priority	Files
Task mode	Execution mode + Plan/Orbit UI	view_only / design_then_view / plan_to_perspective	Separate product intent from transport	P0	novel-view.ts; aiJobService.ts
Authoritative source	Graph target asset	Verified source kind, lineage, content hash	Rebuild on server; no node-ID fallback	P0	graph context; job-image-sources.ts

Capability	Current	Target	Gap / change	Priority	Files
Normalized camera	Raw directive	Versioned pose + projection + frame	Add shared normalizer and adapters	P0	novel-view.ts; new camera-normalization.ts
Scene evidence	Source/reference images only	Evidence manifest with provenance	Add worker SceneEvidenceBuilder	P1	new worker novel-view/
Depth / warp	Absent	Internal depth and coarse RGB guide	Phase 2 z-buffer reprojection	P2	depth; geometry modules
Uncertainty	Absent	Target-space visibility and uncertainty	Coverage/disocclusion map	P2	visibility.ts
Provider conditioning	Direct OpenAI call	ModelConditioning + capability adapter	No provider fields in shared types	P1	model-conditioning.ts; provider interface
Candidates	One per shot	Risk-based 1-4 candidates	Stable candidate IDs and budget	P2	job/API/worker
Reranking	Absent	Deterministic evidence-aware evaluator	Hard gates plus weighted metrics	P2	evaluator/
Retry/idempotency	Job-level and some asset reuse	Per-shot/candidate resume	Persist after each shot; outcome_unknown	P0	generation-service; migration 028
Versioning	Prompt V2 and novel-view-v1	Camera/evidence/adapter versions	Freeze versions in job	P0	shared + repository
Observability	Safe logs and stage errors	Coverage, receipt, scores, per-shot latency	Allowlist non-sensitive metrics	P1	safe-logger; stage errors

6. Minimum viable multimodal input

MVP must have: one canonical source image; normalized target CameraSpec; structured PromptSpec; provider-independent conditioning; deterministic ownership/version hashes. For the first geometry-capable slice, add a coarse RGB target-view guide and target-space uncertainty map.

Phase 2: monocular depth and z-buffer reprojection. Depth is useful internally to make the guide; it does not need to be sent as a native depth tensor to GPT Image.

Not now: full 3D reconstruction, point-cloud persistence, automatic dense semantic masks, browser GPU CV, and paid VLM evaluation.

OpenAI Images Edit accepts multiple images and a mask. The current official API reference documents up to 16 GPT Image input images, n from 1 to 10, and a mask bound to the first image. GPT Image 2 does not expose camera matrices, depth tensors, point clouds, or ControlNet fields. Therefore geometry must be converted to an image guide or mask before this adapter. The native mask is prompt guidance and does not guarantee exact pixel preservation.

7. Production-ready TypeScript contracts

```
export type TaskMode =
  | "view_only"
  | "design_then_view"
  | "plan_to_perspective";

export interface AssetRef { readonly assetId: string }
export interface Confidence {
  readonly value: number | null;
  readonly basis: "measured" | "estimated" | "heuristic" | "unknown";
  readonly method: string;
  readonly version: string;
}
export interface Provenance {
  readonly origin: "user_authored" | "captured" |
    "model_estimate" | "geometric_derivation" | "generated";
  readonly inputAssetIds: readonly string[];
  readonly method: string;
```

```
  readonly version: string;
  readonly assumptions: readonly string[];
}
export interface CameraPose {
  readonly position: readonly [number, number, number];
  readonly target: readonly [number, number, number];
  readonly up: readonly [number, number, number];
}
export interface PerspectiveProjection {
  readonly model: "pinhole";
  readonly horizontalFovDeg: number;
  readonly aspectRatio: number;
  readonly principalPointUv: readonly [number, number];
}
export interface CameraSpec {
  readonly schemaVersion: 1;
  readonly normalizationVersion: string;
  readonly convention: "rh_y_up_camera_minus_z_v1";
  readonly coordinateSpace: "source_relative" | "plan_world";
  readonly pose: CameraPose;
  readonly projection: PerspectiveProjection;
  readonly framingMode: "preserve_subject" | "preserve_footprint" | "custom";
  readonly authored?: {
    readonly azimuthDeltaDeg?: number;
    readonly elevationDeltaDeg?: number;
    readonly distanceRatio?: number;
    readonly focalLengthEquivalentMm?: number;
    readonly rollDeg?: number;
  };
  readonly provenance: Provenance;
}
export type VisibilityClass = "observed" | "inferred" | "unobserved";
export interface AuthoritativeSource {
  readonly image: AssetRef & { width: number; height: number; mimeType: string };
  readonly nodeId: string;
  readonly kind: "site_image" | "design_result" | "plan";
  readonly provenance: Provenance;
}
export interface ReferenceInput {
  readonly contextId: string;
  readonly image: AssetRef;
  readonly role: "layout" | "style" | "material" | "object" | "composition";
  readonly sourceNodeId: string;
  readonly sourceEdgeId: string;
  readonly required: boolean;
  readonly provenance: Provenance;
}
export interface SceneConstraints {
  readonly preserveExactly: readonly string[];
  readonly changeOnly: readonly string[];
  readonly forbiddenChanges: readonly string[];
  readonly protectedRegionIds: readonly string[];
  readonly newlyRevealedPolicy: "conservative_continuation" | "require_observation";
}
export interface GenerationRequest {
  readonly schemaVersion: 1;
  readonly requestId: string;
  readonly shotSetNodeId: string;
  readonly shotId: string;
  readonly shotOrder: number;
  readonly taskMode: TaskMode;
  readonly authoritativeSource: AuthoritativeSource;
  readonly camera: CameraSpec;
  readonly rawGoal: string;
  readonly references: readonly ReferenceInput[];
  readonly constraints: SceneConstraints;
  readonly candidateCount: 1 | 2 | 3 | 4;
  readonly evidenceRequirement: "optional" | "geometry_required";
  readonly provenance: Provenance;
}
export interface SceneEvidence {
  readonly schemaVersion: 1;
  readonly evidenceId: string;
  readonly sourceImage: AssetRef;
  readonly sourceContentHash: string;
  readonly targetCameraHash: string;
  readonly builderVersion: string;
  readonly status: "ready" | "partial" | "unavailable";
  readonly depthMap?: AssetRef;
  readonly depthConfidence?: AssetRef;
  readonly coarseCameraGuide?: AssetRef;
  readonly uncertaintyMask?: AssetRef;
  readonly protectedRegionMask?: AssetRef;
```

```
  readonly semanticMasks?: readonly AssetRef[];
  readonly cameraOverlay?: AssetRef;
  readonly geometryConfidence: Confidence;
  readonly coverage?: { observed: number; inferred: number; unobserved: number };
  readonly degradationCodes: readonly string[];
}
export interface PromptSpec {
  readonly schemaVersion: 1;
  readonly taskType: TaskMode;
  readonly authoritativeSourceDescription: string;
  readonly targetCameraInstruction: string;
  readonly preserveExactly: readonly string[];
  readonly changeOnly: readonly string[];
  readonly newlyRevealedAreas: readonly string[];
  readonly forbiddenChanges: readonly string[];
  readonly outputValidation: readonly string[];
  readonly planHash: string;
}
export interface ModelConditioning {
  readonly schemaVersion: 1;
  readonly conditioningId: string;
  readonly conditioningHash: string;
  readonly taskMode: TaskMode;
  readonly authoritativeSource: AuthoritativeSource;
  readonly targetCamera: CameraSpec;
  readonly sceneEvidence: SceneEvidence;
  readonly referenceImages: readonly ReferenceInput[];
  readonly instructions: PromptSpec;
  readonly metadata: {
    readonly promptCompilerVersion: string;
    readonly cameraNormalizationVersion: string;
    readonly geometryPipelineVersion?: string;
  };
}
export type ConditioningImageRole =
  | "authoritative_source" | "camera_guide" | "uncertainty_guide"
  | "protected_region" | "reference";
export interface ProviderGenerationRequest {
  readonly schemaVersion: 1;
  readonly invocationId: string;
  readonly conditioning: ModelConditioning;
  readonly orderedImages: readonly {
    readonly role: ConditioningImageRole;
    readonly asset: AssetRef;
    readonly required: boolean;
  }[];
  readonly candidateIds: readonly string[];
  readonly output: { width: number; height: number; aspectRatio: number };
}
export interface GenerationCandidate {
  readonly candidateId: string;
  readonly shotId: string;
  readonly candidateIndex: number;
  readonly invocationId: string;
  readonly conditioningHash: string;
  readonly image: AssetRef;
}
export interface EvaluationScore {
  readonly schemaVersion: 1;
  readonly candidateId: string;
  readonly evaluatorVersion: string;
  readonly decision: "accept" | "reject" | "needs_review";
  readonly compositeScore: number | null;
  readonly hardGateFailures: readonly string[];
  readonly metrics: readonly {
    name: string; rawValue: number | null; normalizedScore: number | null;
    confidence: Confidence; supportFraction: number;
  }[];
}
```

All shared contracts use stable asset IDs and provenance. Provider-specific fields such as FormData, n, endpoint URLs, ControlNet scales, or file upload IDs belong only in an adapter.

8. Camera and evidence pipeline

Canonical convention: right-handed world frame, +X right, +Y up, camera forward -Z, image v downward, angles in degrees. Plan legacy Z-up can be adapted to canonical Y-up with an explicit handedness-preserving transform. Virtual distances must

be labeled relative until calibrated.

Required helpers: normalizeCameraSpec(), adaptLegacyCameraShot(), normalizeAzimuthDeg(), focalLengthToHorizontalFov(), horizontalToVerticalFov(), cameraPoseToMatrices(), projectWorldToImage(), orbitCamera(), estimateCameraRisk(), buildCameraGuide().

SceneEvidenceBuilder should first validate orientation, dimensions, source hash, camera assumptions and optional overlay. The next slice adds a worker-only depth adapter, z-buffer reprojection, coverage and disocclusion detection. The browser should remain an authoring/preview surface.

Coarse warp algorithm: decode and orient source; estimate relative or metric depth; unproject source pixels; transform source camera to world and target camera; project into target pixels; resolve collisions with a z-buffer; write RGB guide, coverage and uncertainty. Holes are unobserved, not silently filled as observed.

Intermediate assets: PNG for RGB guide and masks; Float32 little-endian depth only when persistence is needed; source/target hashes and algorithm versions on every artifact. Cache by source content hash + camera hash + output frame + pipeline version, never by signed URL.

9. Uncertainty and evaluator design

Class	Meaning	Use
observed	Target pixels supported by direct source projection and acceptable confidence	Observed-region structure metrics; high preservation weight
inferred	Continuation, interpolation, or low-confidence projection	Guide and conservative generation instruction; lower metric weight
unobserved	Disocclusion, invalid depth, outside frustum, or no source support	Risk gate; never reward decorative completion

MVP evaluator should be deterministic and evidence-aware: output integrity, projected landmark displacement, protected-region overlap where masks exist, guide leakage, camera/framing proxy, and observed-region structure similarity. A hard gate failure rejects a candidate. An unavailable metric stays unavailable; it is never converted to a false zero or one.

Composite score = sum(weight x normalizedScore x confidence) / sum(weight x confidence), with explicit supportFraction. Cross-view consistency comes later and compares only shared observed surfaces or projected landmarks.

10. File-by-file implementation plan

File	Action	Change	Reason
packages/shared/src/novel-view.ts	ADD	TaskMode, normalized camera versions, legacy adapter types	Shared request authority
packages/shared/src/model-conditioning.ts	ADD	SceneEvidence, ModelConditioning, roles, candidates, scores	Provider-neutral boundary
packages/shared/src/camera-normalization.ts	ADD	Canonical frame, pose/projection math, FOV/lens conversion	One camera meaning
packages/ai/src/prompt-engine/compileProviderPrompt.ts	REFACTOR	Explicit Orbit/Plan dispatch; render approved semantic plan	Fix F01-F03
packages/ai/src/prompt-engine/orbitPromptCompiler.ts	ADD	Orbit PromptSpec with raw goal, operations, roles, constraints	Preserve semantics
packages/ai/src/prompt-engine/planPromptCompiler.ts	ADD	Plan target-height and footprint semantics	Remove generic fallback
apps/web/lib/server/aiJobService.ts	FIX	Canonical hash, retry ordering, candidate budget	Idempotency and cost
apps/worker/src/services/generation-service.ts	REFACTOR	Per-shot execution, decision gate, persist-before-next-shot	Safe replay
apps/worker/src/providers/image-provider.ts	ADD	Provider interface and capabilities	Adapter isolation

File	Action	Change	Reason
apps/worker/src/providers/openai/generate-image.ts	FIX	Resolved model, role ordering, mask binding, receipt	Correct OpenAI call
apps/worker/src/novel-view/scene-evidence-builder.ts	ADD	Source evidence and versioned builder	Multimodal foundation
apps/worker/src/novel-view/geometry/reproject.ts	ADD Phase 2	Depth-aware z-buffer warp	Geometry MVP
apps/worker/src/novel-view/geometry/visibility.ts	ADD Phase 2	Observed/inferred/unobserved and protected projection	Uncertainty control
apps/worker/src/novel-view/evaluator/evaluate-candidate.ts	ADD Phase 2	Deterministic metrics and hard gates	Reranking
packages/db/sql/028_multi_angle_execution_manifest.sql	ADD later	Per-shot invocation/candidate state, CAS, RLS, indexes	Durable resume

11. Task list

MA-001 - Canonical source and request identity (P0)

Files: Graph context, multiAngleRequestService.ts, aiJobService.ts

Acceptance: Asset-ID-only canonical hash; source rebuilt and ownership checked; URL refresh does not change logical request.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-002 - Explicit Orbit/Plan prompt compilers and decision gate (P0)

Files: compileProviderPrompt.ts, Orbit/Plan compilers, generation-service.ts

Acceptance: No semantic loss; Plan fields affect prompt; reject/review makes zero provider calls.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-003 - Versioned CameraSpec normalization (P0)

Files: novel-view.ts, camera-normalization.ts, API schema

Acceptance: Deterministic normalization, 360-degree roundtrip, no NaN, explicit virtual scale.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-004 - Carry normalized camera through request/job/snapshot (P0)

Files: shared jobs/snapshot, hydration, repository

Acceptance: UI to worker roundtrip; legacy snapshots still load.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-005 - Fix terminal rerun and per-shot replay (P0)

Files: aiJobService.ts, generation-service.ts

Acceptance: Second-shot failure does not regenerate first; no duplicate credit debit.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-006 - ModelConditioning assembly (P1)

Files: model-conditioning.ts, build-model-conditioning.ts

Acceptance: Stable conditioning hash; no provider-specific fields.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-007 - Provider role mapping and OpenAI model forwarding (P1)

Files: provider interface, OpenAI adapter, job-image-sources.ts

Acceptance: Bytes and roles match manifest; actual model equals recorded model.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-008 - Durable per-shot invocation state (P1)

Files: migration 028, shot executor, asset persistence

Acceptance: Crash/replay resumes without duplicate candidate calls; unknown outcomes are explicit.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-009 - Source evidence and camera overlay (P1)

Files: scene-evidence-builder.ts, camera-overlay.ts

Acceptance: Deterministic source artifact and honest degradation state.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-010 - Depth adapter and coarse reprojection (P2)

Files: depth/, geometry/

Acceptance: Identity camera reconstruction tolerance; foreground occlusion and holes correct.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-011 - Deterministic evaluator and candidates (P2)

Files: evaluator/, API/jobs, shot executor

Acceptance: Risk-based 1-4 candidates, hard gates, deterministic winner, no paid VLM.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

MA-012 - Cross-view consistency and alternative providers (P3)

Files: family evaluator, new adapters

Acceptance: Shared observed landmarks and provider benchmark with same conditioning.

Tests: unit, golden, synthetic geometry, fake-provider integration, and replay fixture as applicable.

12. Migration phases

Phase	Scope	Exit condition
0	Fix semantics, TaskMode, CameraSpec, Orbit/Plan PromptSpec, tests	No prompt semantic loss; camera survives request path

Phase	Scope	Exit condition
1	ModelConditioning, provider capabilities, image roles/order, overlay	OpenAI adapter consumes a role manifest without shared provider fields
2	Depth adapter, coarse warp, uncertainty/protected masks	Synthetic geometry and evidence gates pass; fallback is explicit
3	Per-shot candidates, durable manifest, deterministic evaluator/rerank	No duplicate paid calls on replay; winner is reproducible
4	Cross-view metrics and specialized multi-view provider experiments	Benchmarked against the same conditioning/evaluator

13. Delete/refactor decisions

• **Keep:** graph inbound context, stable asset gateway, queue/outbox, operation-first persistence, ratio registry, PromptPlanV2, fake transports.

• **Refactor:** camera math into shared helpers; prompt rendering into explicit compilers; generation service into a resumable shot executor.

• **Deprecate:** Orbit-only ImageGenerationRequest as an execution boundary; ambiguous viewDirection; virtual distance labeled as measured metres.

• **Remove after compatibility tests:** Orbit early return, Plan generic fallback, batch-all-before-persist, node-ID image fallback, silent reference truncation, retry-key-after-RPC branch.

14. MVP boundary

MUST HAVE: canonical source; normalized CameraSpec; explicit Orbit/Plan PromptSpec; provider-neutral conditioning; image role ordering; per-shot durable identity; deterministic eligibility and replay tests.

NICE TO HAVE: camera overlay, two candidates for moderate risk, user landmarks, protected region projection, source-depth cache.

NOT NOW: full 3D, Blender, exact unseen-backside reconstruction, dense auto-segmentation, browser CV, paid VLM gating, joint multi-view training.

15. Recommended first PR

PR title: fix(ai): preserve multi-angle semantics with explicit camera compilers

• Modify shared novel-view/prompt types and the provider prompt compiler.

• Add OrbitPromptCompiler and PlanPromptCompiler.

• Preserve raw goal, approved operations, reference roles and hard constraints.

• Make Plan targetHeight and viewDirection semantically effective.

• Block provider calls for reject or require_review decisions.

• Add golden prompt tests, sentinel semantic-loss tests, Plan precedence tests, and fake-provider zero-call tests.

Definition of done: existing snapshots and ordinary Image Generator behavior remain compatible; focused AI/worker tests pass; no real provider call; no depth dependency; `git diff --check` and relevant typechecks pass. This PR creates the semantic seam for ModelConditioning without attempting full geometry.

16. Verification record and references

Repository verification: canvas workflow tests 28 passed; prompt engine tests 16 passed; worker image-generator tests 15 passed; git diff --check passed; worktree had no changes before PDF creation. Full repository build/lint/typecheck, staging migration, E2E, and real-image quality benchmarks were not run for this audit.

Official OpenAI documentation used for the provider capability audit: GPT Image 2 model page, Image Generation guide, and Create Image Edit API reference. The API reference documents multiple image inputs, image ordering semantics for masks, output count, supported sizes, and the lack of native camera/depth fields.

External technical references used for implementation guidance: OpenCV pinhole projection and extrinsics/intrinsics documentation; Depth Anything V2 repository for a possible later depth-estimator benchmark. These references do not change the MVP boundary.